Technische Hochschule Ingolstadt

(Forecasting for Engineering and Management)

From Joint Pain To Digital Gain: Time Series Analysis of Knee Recovery & Bitcoin Prices

Author: **Wajeeh Ul Hassan Malik**
ID: 00137328
E-mail: *wam9284@thi.de*

June 24, 2025

Abstract

This report dives into two very different time series. One from a recent injury and one from the world of cryptocurrency. In Part 1, I recorded my post-op pain scores tri-daily to analyze trends, seasonality and recovery progression. The data showed a clear downward trend and daily cycles, though it required differencing for proper modeling. In part 2, I switched gears to the chaotic yet fascinating behavior of Bitcoin prices. Using a dataset spanning over 10 years, I compared ARIMA, Prophet and LSTM models to forecast daily closing prices. While ARIMA fell short, Prophet caught some trends and LSTM delivered the best performance by far. This project blends personal health tracking with financial forecasting, showing how time series models can adapt across domains, from joints to coins.

Contents

1 Part 1: Self-Collected Data Analysis

1.1 Introduction

Rationale for Dataset Choice

I chose to collect and analyze my own pain level data because the experience of recovering from surgery was very personal, yet it had a clear structure that could benefit from time series analysis. My immobility during recovery and the importance of managing pain effectively made this a topic with both emotional importance and real world relevance. In fact, my professor even pointed out that this could be an ideal case of self-quantification, transforming a subjective and internal experience into data. Using this dataset allows me to explore whether pain follows any trends cycle or daily pattern, and how these could be forecasted to evaluate the effectiveness of my recovery strategies. Also, since this kind of pain tracking could be helpful in healthcare or rehab management, it seemed meaningful to investigate it using forecasting tools.

Dataset Description

After undergoing knee surgery on May 27th, 2025, I began tracking my post-operative pain levels using a numerical rating scale from 0 to 10, where 0 represents no pain and 10 represents the worst pain imaginable. I recorded my pain levels three times per day (at 09:00, 15:00, and 21:00 CEST), starting from May 28th to June 16th, 2025, which gave me a total of 60 pain level entries across 20 days. Each observation captures how intense my pain felt at that specific time of the day, allowing me to observe fluctuations throughout the day and over the course of my recovery.

1.2 Data Collection Method

1.2.1 Procedure

To ensure consistency, I developed a simple daily routine: I recorded my pain at three fixed times each day (morning 09:00, afternoon 15:00, and night 21:00). I started this on May 28th, the day after my operation, and continued it until June 16th, giving me a complete record across 20 days i.e. 60 entries in total. The idea was to capture how my pain changed throughout the day and over time. I made sure to follow the schedule strictly, even setting reminders to avoid missing entries. The pain scores were recorded on a 0-10 scale, including both whole numbers and decimal values (e.g. 7, 6,5, 8) depending how I felt at that moment. This allowed me to express pain more precisely.

1.2.2 Tools & Devices used

I relied primarily on my smartphone to help with this task. I used the built-in calendar and reminder apps to set alarms for each of the tri-daily recording times.To determine my pain score, I referred to a standard numerical pain scale [1], which typically uses whole numbers from 0-10. However, in my recordings, I frequently included decimal values (like 6,5 or 7,5) when the pain felt somewhere in between two levels. While this goes on slightly beyond the chart's original format, I found it gave a more accurate reflection of how I

was feeling. All values were logged in a Microsoft Excel spreadsheet, which helped keep things organized and easy to analyze later in Python.

1.3 Descriptive Analysis

1.3.1 Basic Statistics & Plot Analysis

Mean	Median	Standard Deviation	Minimum	Maximum
6.00	6.00	1.53	3.00	9.00

Table 1: Descriptive Statistics of Pain Level (1-10 Scale)

From the summary table 1, we can see that both the mean and median pain levels were 6.00, which suggests fairly symmetrical distribution. The standard deviation of 1.53 indicates relatively low variability, meaning most of my pain stayed near that average. Pain levels ranged from 3 (minimum) to 9 (maximum), covering a wide range but nothing that distorts the average.

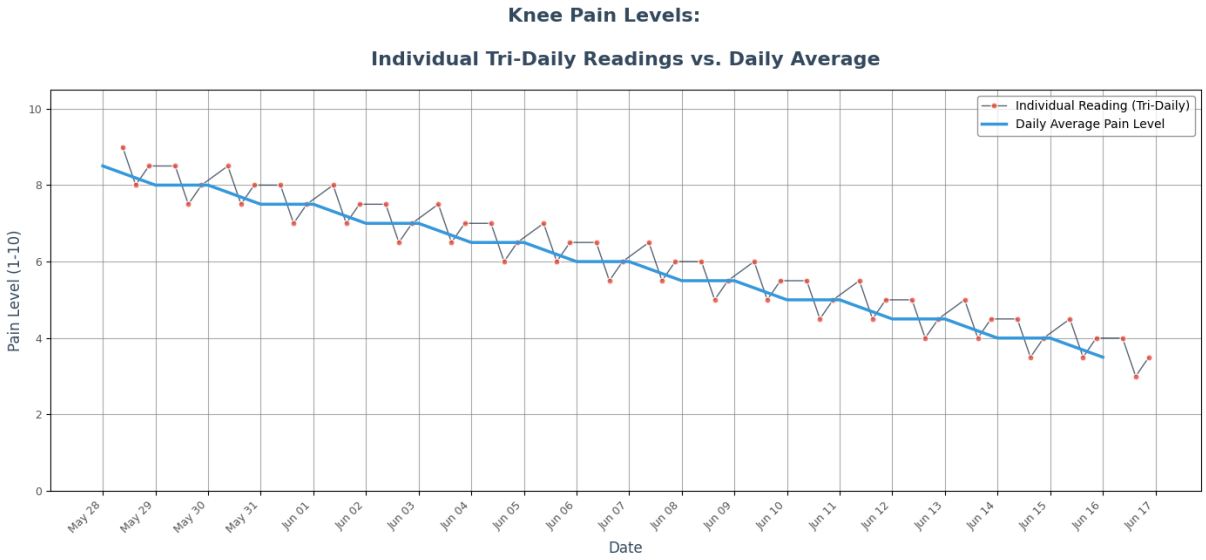

Figure 1: Time series pain level plot

When visualizing the time series in Figure 1, a clear downward trend in daily pain averages is visible. Despite some minor ups and downs, the overall pattern shows steady recovery. There were no missing values in the dataset, I verified this using [df.isnull().sum()] in Python, which returned all zeroes. Also, there were no major outliers; any spikes or dips seen in the plot fall within normal post-op fluctuation and are explainable based on recovery.

1.3.2 Serial Dependence & Temporal Structure Analysis

To explore how my pain levels depend on previous values, I observed Autocorrelation & Partial Autocorrelation Function plots,these are really helpful for highlighting any hidden patterns or dependencies in the data.

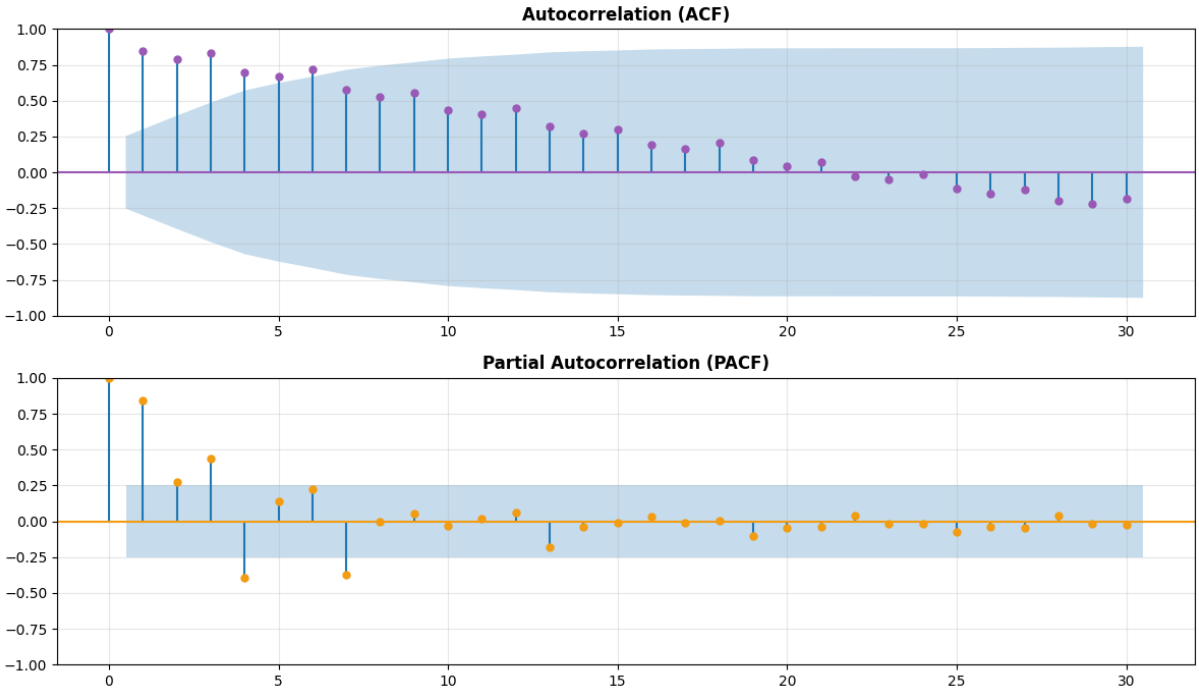

Figure 2: ACF & PACF plots

In the **ACF plot**, we see a gradual decline across many lags,suggesting the presence of a trend in the data; likely due to my gradual recovery. Interestingly, there are repeating spikes at lags 3,6,9 etc., which lines up with my tri-daily recording schedule (every 3 entries = 1 day). This implies a daily seasonality, my pain at 09:00 today is often similar to the pain I had at 09:00 yesterday.

The **PACF plot** shows a strong spike at lag 1, followed by values dropping within the confidence interval (blue shaded region). This suggests that the pain level at any time is most directly influenced by the previous reading, and the impact of older lags is limited.

1.3.3 Pattern Identification, Decomposition, & Stationarity Check

Before applying any forecasting models, its important to check if the time series is stationary, meaning its average (mean) and variation (variance) stay constant over time. A non-stationary series often has trends or seasonality that need to be handled first. By visually inspecting the Pain level plot (Figure 1), there's a clear downward trend, hinting that the data likely isn't stationary; my average pain is gradually decreasing as I recover. To confirm I performed Seasonal Decomposition with a period of 3, since I recorded data three times a day.

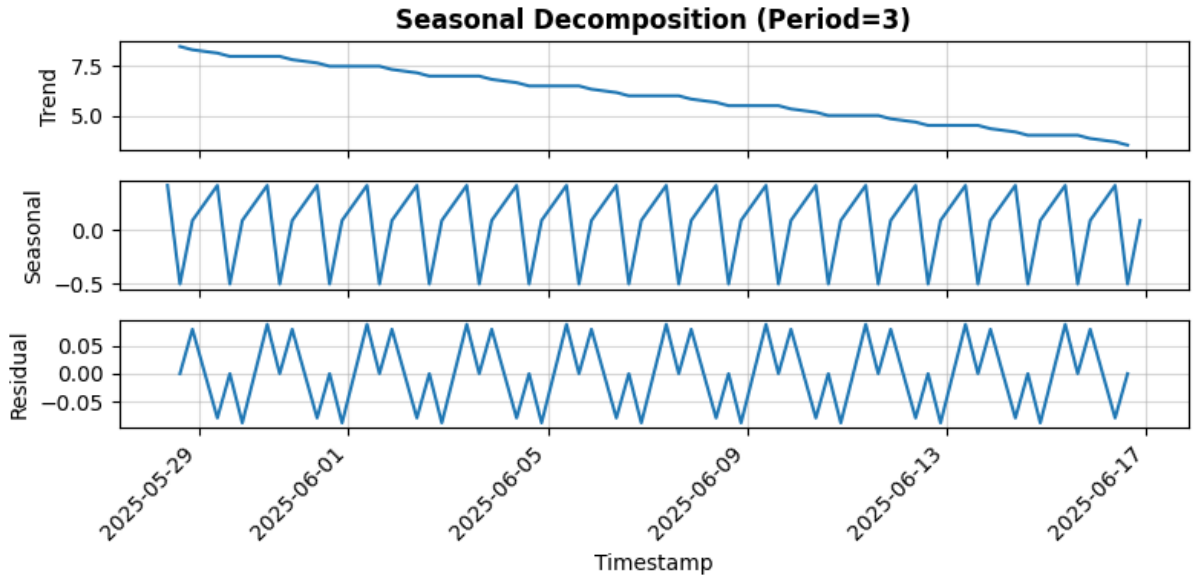

Figure 3: Tri-Daily Seasonal Decomposition of Post-Operative Pain Scores

Trend: There's a steady decrease, which matches the overall downward pattern in my pain levels.

Seasonal: Shows a repeating cycle across 3 readings per day. This fits the idea that my pain levels often behave similarly at the same time each day (e.g., mornings).

Residual: It appears random with no strong pattern, which is ideal after removing trend and seasonality.

To formally test for stationarity, I ran the Augmented Dickey-Fuller (ADF) Test. The results are:

Table 2: Augmented Dickey-Fuller Test Results

ADF Statistic	**p-value**
-0.0203	0.9568

Since the p-value is much greater then 0.05, I failed to reject the null hypothesis, which confirms the series is non-stationary. This means before using a model like ARIMA, I will likely need to apply differencing or transformation to make the data more stable.

1.4 Discussion

1.4.1 Interpretation of Basic Data Properties

Looking at the basic summary of my pain levels, I noticed that both the average and median were exactly 6, which means that overall, the pain levels were pretty balanced, not too skewed to one side. The standard deviation was only 1.53, which tells me that my pain didn't jump around a lot and stayed close to that average most of the time. Also, the values ranged from 3 to 9, so while I did feel both low and high pain, there weren't any entries that looked like a mistake or outliers. I made sure there were no missing values during my data collection, and the graph didn't show anything that looked strange or unexpected either. This gives me confidence that the dataset is clean and reliable for modeling.

1.4.2 Interpretation of Temporal Structures

Since I recorded pain levels tri-daily, I was curious if there were any patterns. The ACF plot showed spikes at lag 3,6,9 and so on; which is one full day, two days , etc. This clearly shows that my pain at a certain time (e.g.; morning) was similar to the pain at the same time the day before.

This makes sense, because after surgery, my pain didn't change randomly, it kind of followed a routine, for example, mornings were often stiffer or more uncomfortable, while evenings felt more relaxed after using RICE (Rest, Ice, Compression and Elevation) therapy.

This daily pattern means that any future model should try to capture these repeating effects that happen around the same times each day.

1.4.3 Interpretation of Global Patterns

The biggest thing I noticed in the graph is the downward trend. My pain levels dropped over the 20 days, which matches how I was feeling in real life. The seasonal pattern also showed up (as explained before), repeating roughly everyday. But when I tested for stationarity using the ADF test, it turned out the data was not stationary. The test gave a high p-value (0.95), which means the average pain level was not stable over time; it was dropping, just like we saw in the trend. This matters because many forecasting models like ARIMA assume stationarity, so I will need to either difference the data or transform it before trying to predict anything.

So in short there's a clear trend (improving over time), a daily seasonality and the data is non-stationary therefore requires handling before modeling.

2 Part 2: Public Dataset Analysis

2.1 Dataset Overview and Full EDA

2.1.1 Introduction and Topic Relevance

For this project, I chose to forecast the price of Bitcoin. I picked Bitcoin because its price is famous for being highly unpredictable and volatile, which makes it a really interesting challenge for time series analysis. The dataset I'm using is from Kaggle [2] and it originally had Bitcoin's price in USD for every minute.

After preparing the data, my final time series contains the daily closing price from January 1, 2012 to May 23, 2025. My main goal is to see if I can accurately forecast this daily price, which I think would be a practical and useful forecast. As an example, someone who invests in Bitcoin could use it to get a better idea of where the price might be heading, which might help them when to buy or sell. Analyzing such a famous and tricky dataset is a great way to test out different forecasting models and see how they perform in a real world scenario.

2.1.2 Interpretation of Basic Data Properties

To get a feel for the data, I started by looking at some basic statistics using the [describe()] function in Python.

Table 3: Descriptive Statistics for Daily Close Price (USD)

Statistic	**Value**
Mean	17,805.92
Std Dev	24,702.68
Min	4.38
Median	6,674.74
Max	111,743.00

The numbers in (Table 3) immediately show how erratic the price history is. Over 4,892 days in the dataset, the average price was $17,805.92. However, the standard deviation was higher, at $24,702.68, which points to extreme variability. The price has gone from a low of just $4.38 to a high of $111,743.00.

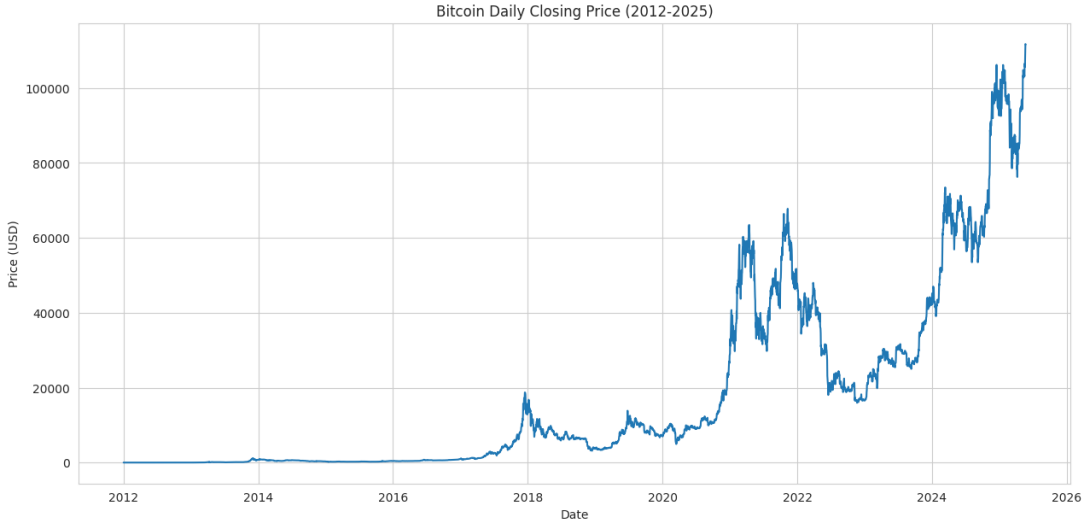

Figure 4: Time series plot for daily closing Bitcoin price

The plot (Figure 4) above of the data over time confirms this perfectly. For the first several years (from 2012 to about 2017), the price was very low and flat. Then you can clearly see several major spikes: a significant one in late 2017/early 2018, another huge one around 2021 and the period of most explosive growth in late 2024 and early 2025. These spikes are not a data error; they are real market events that show just how volatile Bitcoin really is. My code also confirmed there are no missing values to worry about.

2.1.3 Temporal Structure Analysis

Next I wanted to see if there were any regular patterns based on the time of the year or day of the week. I created a couple of box-plots to investigate this:

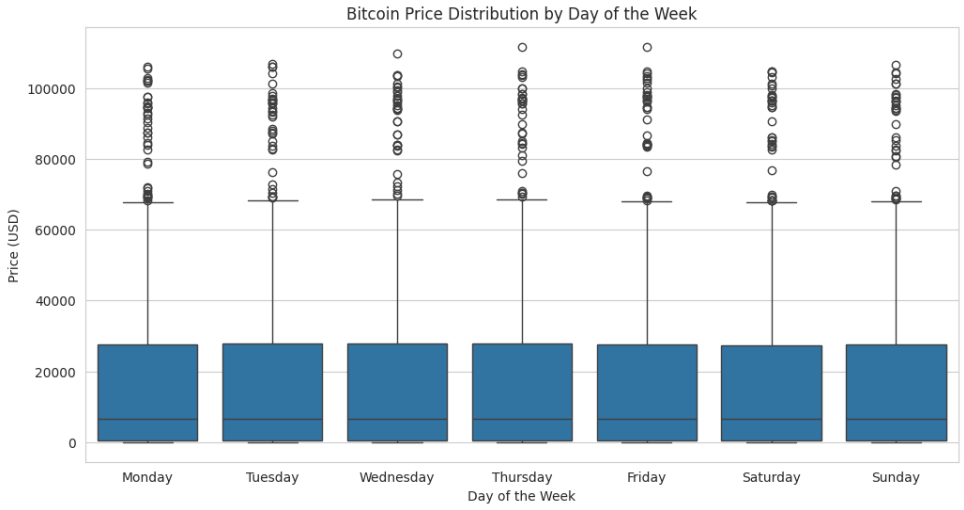

Figure 5: Price Distribution by Weekday

The plot (Figure 5) above, breaks down the price by the day of the week. As you can see from the chart, all the Weekday boxes look very similar. This shows that there is no significant difference in price behavior between, say, a Monday and a Saturday. This is logical, as the Bitcoin market operates continuously, unlike traditional markets that close on weekends.

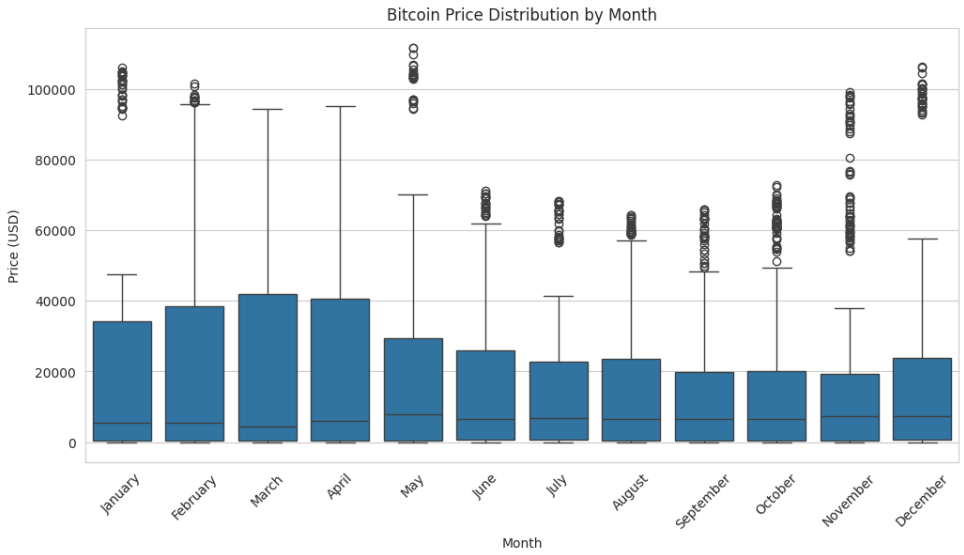

Figure 6: Price Distribution by Day of Month

The plot (Figure 6) shows the price distribution for each month revealing a more interesting potential pattern. It appears that prices are often lower and less volatile during summer months (around June to September). Conversely, the beginning and the end of the year months, like March, April and November, show a higher median prices and more spread-out data, suggesting more volatility. While this is not a very strong seasonal effect, it's a tendency worth nothing.

2.1.4 Pattern Identification, Decomposition and Stationarity Check

To finish off my initial analysis, I did a more technical check of the data's global patterns, focusing on trend, seasonality and stationarity.

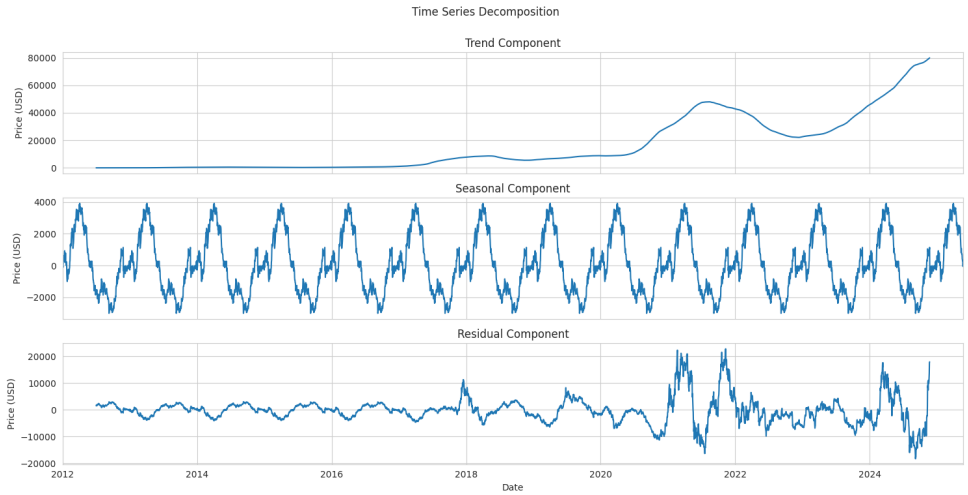

Figure 7: Decomposition Plots BTC price

First, I used a seasonal decomposition plot (Figure 7) to break the series down. This was helpful because it visually separated the trend, seasonal and residual components. The "Trend" part of the plot was the most obvious, clearly showing the long-term price movements up and down over the years. I also saw a repeating "Seasonal" pattern, which looks like it had a yearly cycle, but it was much weaker compared to the strength of the main trend.

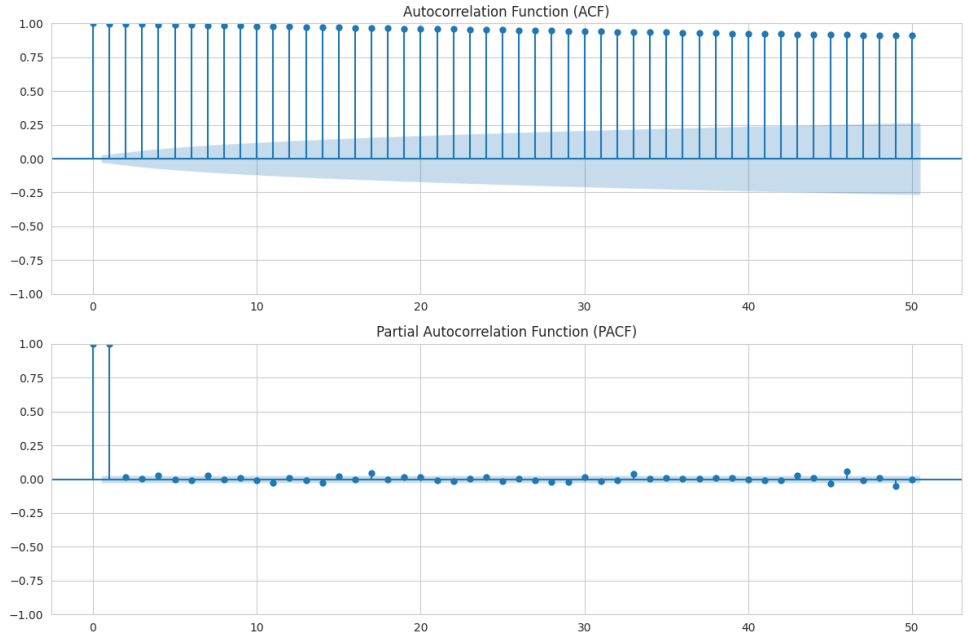

Figure 8: ACF and PACF of Bitcoin Daily Closing Prices

Next, I looked at the Autocorrelation Function (ACF) and The Partial Autocorrelation Function (PACF) plots (Figure 8) to understand the data's autocorrelation. The ACF plot was very revealing. The correlation bars stayed very high and went down very slowly, which is a classic sign of a non-stationary series with a strong trend. The PACF plot was also useful, as it had a large spike at lag 1 and then dropped off. My interpretation of this is that yesterday's price is very strong predictor of today's price, but once you account for that, the prices from earlier days don't add as much new information.

To be absolutely certain about the stationarity, I performed an Augmented Dickey-Fuller (ADF) test.The key output is the p-value.

Table 4: Augmented Dickey-Fuller Test Results

Statistic	**Value**
ADF Statistic	0.6845021914996269
p-value	0.9895234551204807

The result I got was a p-value of 0.9895. Since this is above the 0.05 significant level, it confirmed my visual analysis from the plots: the Bitcoin price series is non-stationary. This was a critical finding for my project, as it meant that I would definitely have to apply differencing to the data before using a model like ARIMA

All of this analysis points to one critical conclusion for my project: any model I use must account for the strong trend and non-stationarity. For statistical models like ARIMA, this means that I will absolutely need to difference the data to make it stationary before I can fit the model.

2.2 Methodology

2.2.1 Model Selection

After exploring the data, the next step was to choose the forecasting models for this project. The decision was to select three different types of models to compare a classical statistical approach, a more modern automated tool, and a complex deep learning method. The chosen models were SARIMA, Prophet, and LSTM network.

The SARIMA model was selected because it seemed like a natural fit based on the initial analysis. Its 'Integrated' (I) component handles differencing, which was necessary since ADF test showed the data was non-stationary. The 'AR' (Autoregressive) and 'MA' (Moving Average) parts are meant to handle the kind of autocorrelation seen in the ACF and PACF plots. The 'S (Seasonal) part could theoretically handle the yearly seasonality found in the data, though this turned out to be difficult to implement in practice.

The second choice, Prophet, is a library developed by Facebook. Prophet was picked because it's designed to automatically handle many of the features found in the data, especially the strong, non-linear trend and multiple seasonalities (weekly and yearly). It was considered interesting to compare the more manual SARIMA approach with Prophet's more automated forecasting process.

The third and the final model was Long Short-Term Memory (LSTM) network. This deep learning model was chosen based on the suspicion that Bitcoin price had complex, non-linear patterns that the other two models might miss. The residual plot from the earlier decomposition showed that the errors were not random, which suggested that more patterns remained to be found. The LSTM was selected to see if a powerful model could learn these patterns from sequences of past data and produce a more accurate forecast.

2.2.2 Data Preparation and Modeling Assumptions

Before training the models, the data was needed to be prepared. First, the dataset was split into a training set and a testing set, using an 80/20 ratio. The first 80% of the data, from January 1st, 2012, to September 17th, 2022, was used as the training data for models. The remaining 20% of the data , a total of 979 days from September 18th,2022, to May 23rd, 2025, was held out as the testing set. This method ensures that the model is tested on "unseen" future data, which simulates a real world forecasting scenarios.

Each model then required specific data preparation. For ARIMA model, the main assumption is that the time series must be stationary. It was clear from EDA that differencing had to be applied. For Prophet, little preparation was needed beyond renaming the columns to 'ds' and 'y', as the library requires. Its designed to handle non-stationary data on its own. The LSTM model required the most preparation. First, all price data had to be scaled to a range between 0 and 1. Then, the data had to be converted into overlapping sequences, where a sequence of 60 previous days was used to predict the next day.

2.3 Results

2.3.1 Model Implementation and Forecasting

In this section, all three models were implemented to generate forecasts.

The implementation began with the SARIMA family model, as planned in methodology. The initial intention was to fit a full seasonal model to capture the yearly patterns found in the data. However, this approach proved to be computationally infeasible in the project environment; the model fitting process didn't complete even after running for over 2 hours. Therefore , a strategic decision was made to opt simpler non-seasonal ARIMA (1,1,1) model. This model was then used to establish practical baseline that could effectively capture the primary trend, although it meant that the seasonal component would be ignored for this specific model.

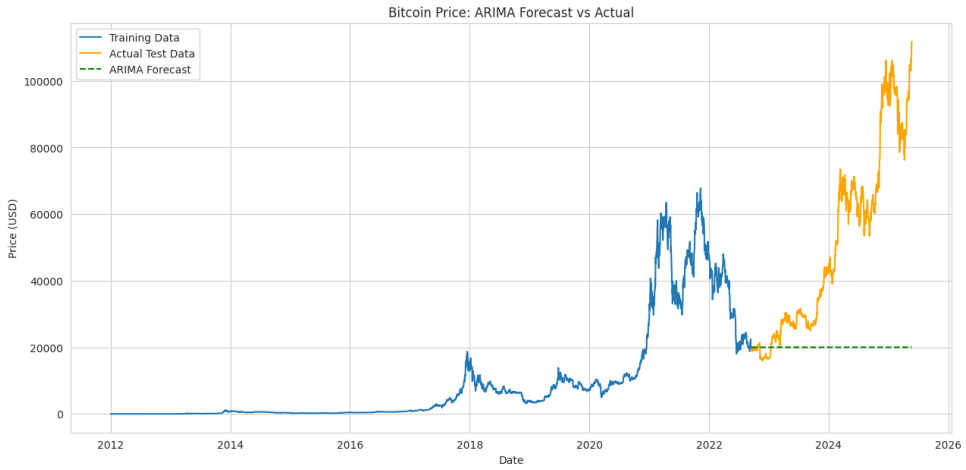

Figure 9: Prophet Forecast vs Actual

The forecast plot (Figure 9) for the ARIMA model showed essentially a flat line projected forward from the last known price. This outcome immediately highlighted that this simple model failed to capture the sharp upward trend in the test data, establishing it as a clear but inaccurate, baseline.

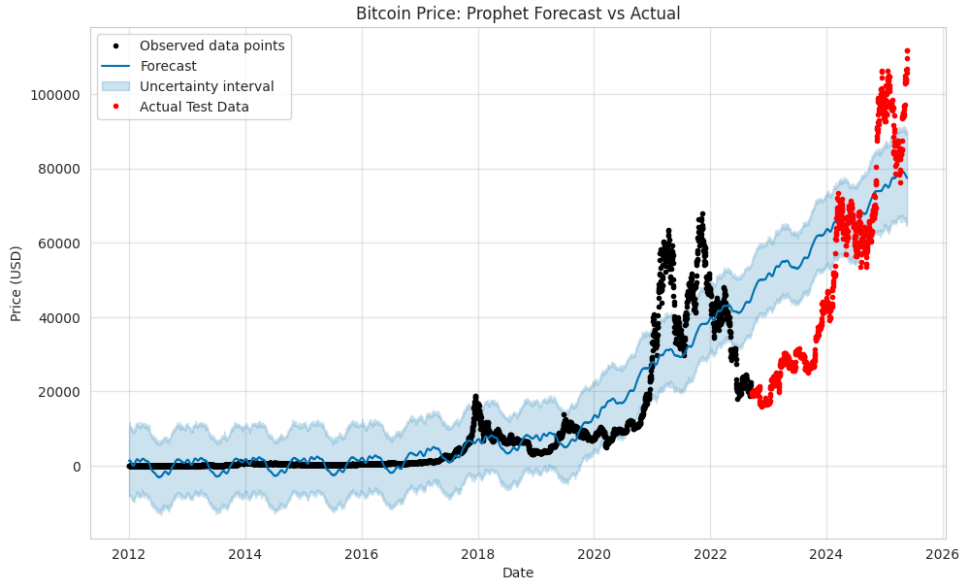

Figure 10: Prophet Forecast vs Actual

Next,the Prophet model was implemented. This produced a significant improvement. Its forecast plot (Figure 10) successfully captured the general upward direction of the price, making it a much more reasonable prediction.

Finally, the LSTM network was built and trained. The results from this model were very impressive. The plot (Figure 11) showed that the LSTM forecast was able to track the actual test price with high accuracy following not just the main trend but also many of the sharp, volatile price swings.

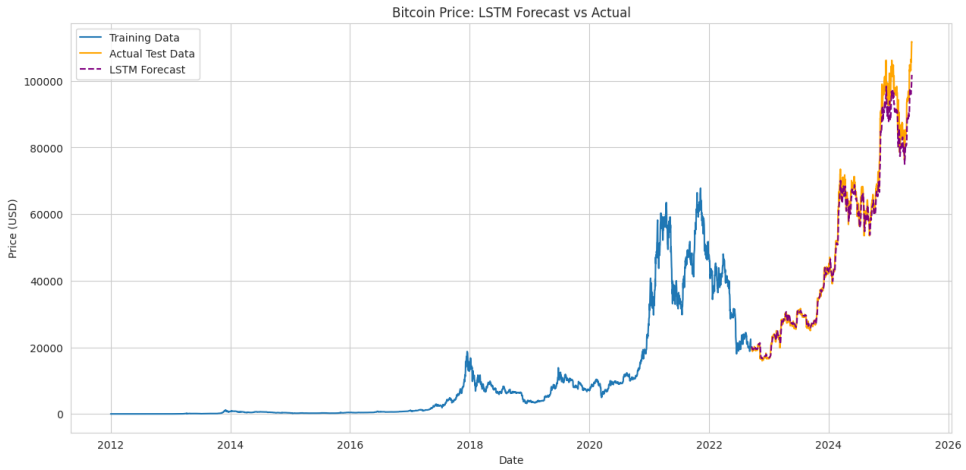

Figure 11: LSTM Forecast vs Actual

2.3.2 Forecast Evaluation and Diagnostics

To evaluate the forecating performance of the models, I used Mean Absolute Error (MAE), as it provides an intuitive and direct measure of prediction accuracy in the same units as the data, in my case US dollars. Among the three models tested LSTM model produced the lowest MAE at Approximately $1,946.31, followed by Prophet at $19,286.26. The ARIMA model , which was computationally heavy and difficult to tune effectively on such a large dataset, had the highest error of $31,453.12. These results were consistent with the visual inspection of the forecast plots, where LSTM followed the test data more closely than other models.

Beyond the MAE scores, diagnostic checks (Figure 12 & 13) provided further insight into each model's behavior. A residual analysis of the ARIMA model revealed that while it successfully capturing the linear autocorrelation in the training data, its errors showed a non-constant variance, indicating a failure to model market's changing volatility. In contrast, Prophet's diagnostic component plot was highly informative, visually confirming that the model had effectively isolated and fitted the non-linear trend and the distinct yearly and weekly seasonal cycles. For LSTM model, the forecast's exceptionally close tracking of the test data serves as a powerful visual diagnostic, suggesting that the model successfully learned the complex sequential patterns without significant error remaining.

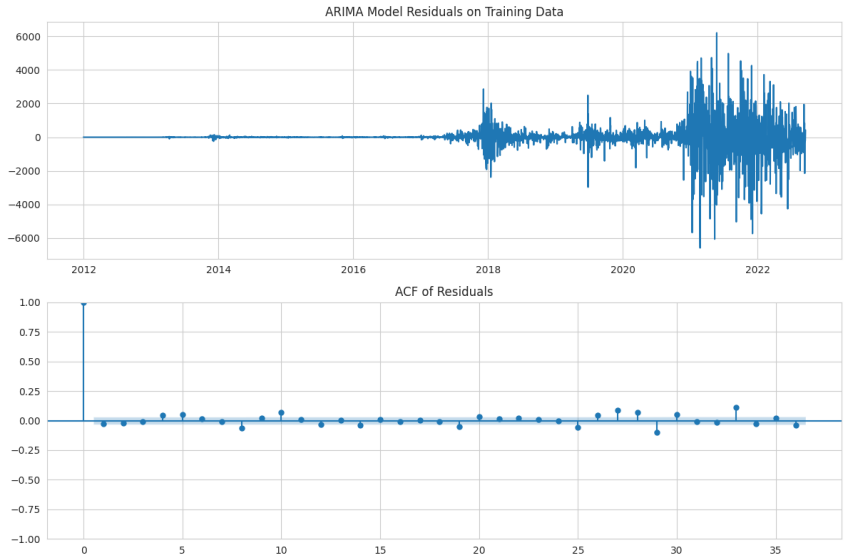

Figure 12: ARIMA Model Residuals and Autocorrelation of Residuals

Figure 13: Trend, Weekly, and Yearly Components Extracted by Prophet

2.3.3 Model Comparison

To make the performance difference between the models easy to see, I summarized their final Mean Absolute Error scores in the table below.

Table 5: Comparison of Mean Absolute Error (MAE) Across Models

Model	**Mean Absolute Error (MAE)**
ARIMA (Baseline)	$31,453.12
Prophet	$19,286.26
LSTM	**$1,946.31**

This clearly highlights how each model improved upon the previous one, with LSTM being the most accurate by a very large margin.

2.4 Discussion

2.4.1 Interpretation of Forecast Performance

My interpretation of the Forecasts is that the complexity of the model was very important for this particular problem. The ARIMA model, which is linear, was too simple to understand the complex trend of the Bitcoin price. Prophet did much better because its main feature is being able to fit strong, non-linear trends. Therefore, it produced a relatively smooth forecast. I believe that the LSTM was the clear winner because its network structure allowed it to learn from sequences of recent data. This helped it to understand not only the trend but also the short-term momentum and volatility, which is why its forecast was so much more detailed and accurate.

2.4.2 Reflection on Modeling Process and Limitations

I ran into a few challenges during this project. The biggest was when my original plan to use the full seasonal SARIMA model failed because it was too computationally slow. It ran for hours and never ended, forcing me to change my course and instead use a simpler ARIMA model as a baseline instead. This was a good lesson in how a model that looks good in theory might not always be practical.

I also think that my final LSTM model has an important limitation, even though its MAE was very low. I believe that its high accuracy comes from being very good at predicting the next day's price based on the momentum of the last 60 days. This means that it is great at tracking the price, but it probably won't be able to predict a sudden market crash or a big change in the overall trend far in advance.

2.4.3 Real World Applications

For a real-world application, such as for an investor, one would need to be very cautious with these forecasts. Even with the LSTM's accurate-looking prediction, it would be extremely risky to use it as a simple "buy" or "sell" signal. Because it's so good at tracking recent momentum, it wouldn't be able to predict a sudden event that changes the market trend. A more realistic use would be as a confirmation tool. For instance, if an investor already thinks the market is in an uptrend, the LSTM forecast could provide extra evidence to support their hunch. It is a powerful tool for analysis, but it's not a substitute for a complete, risk-managed investment strategy.

2.5 Conclusion

This project was a valuable, hands-on experience in applying different forecasting models to the challenging, real world data of Bitcoin's price. Comparing ARIMA, Prophet and LSTM highlighted their unique strengths and weaknesses. I learned the importance of adapting my strategy when faced with practical challenges, such as when the initial SARIMA model proved too slow to run. Overcoming these issues and learning to interpret the results deepened my understanding of key factors of forecasting.
Overall, this analysis shows that while advanced models like LSTMs can provide powerful insights into the market trends, they are not fortunetellers. In a real-world setting, such forecasts should be used with caution and as a part of larger analytical strategy for making financial decisions.

References

[1] H. Breivik et al. "Assessment of pain". In: *BJA: British Journal of Anaesthesia* 101.1 (May 2008), pp. 17–24. DOI: 10.1093/bja/aen103. URL: https://doi.org/10.1093/bja/aen103 (visited on 06/19/2025).

[2] Mikołaj Zieliński. *Bitcoin Historical Data*. 2022. URL: https://www.kaggle.com/datasets/mczielinski/bitcoin-historical-data (visited on 06/23/2025).